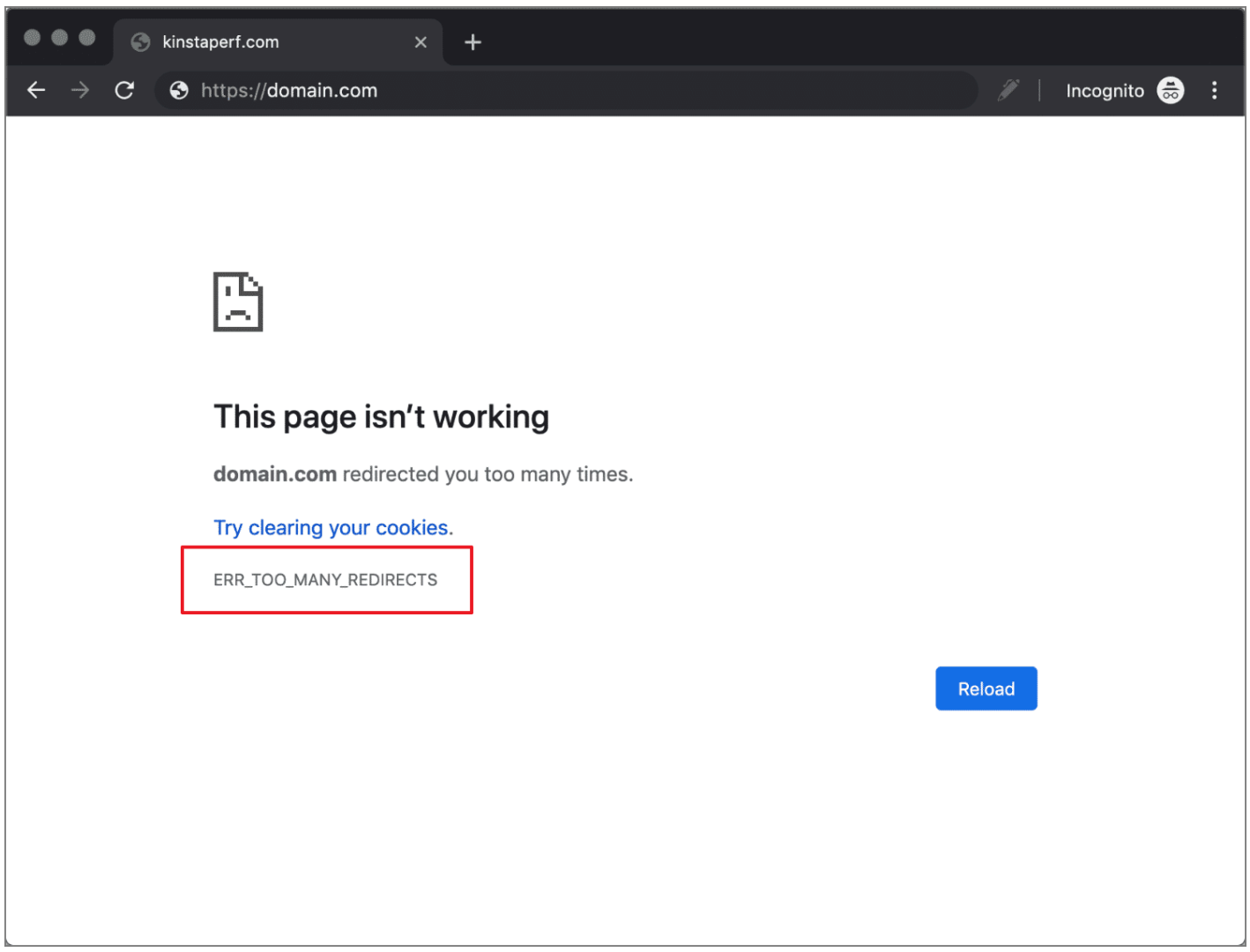
리다이렉트 루프는 왜 발생할까? HTTPS 설정부터 쿠키와 CDN 캐시까지

리디렉션은 브라우저에게 다른 주소로 다시 요청하라고 알려주는 기능이다. 그런데 안내받은 주소로 이동했는데도 또 다른 주소로 가라는 응답을 받고, 결국 같은 곳으로 돌아오는 경우가 있다.

이런 이동이 반복되어 페이지에 도착하지 못하는 상황을 **리다이렉트 루프(Redirect Loop)**라고 한다. 브라우저는 무한히 요청하지 않고, 반복을 감지하거나 리디렉션 횟수 제한에 도달하면 `ERR_TOO_MANY_REDIRECTS` 같은 오류를 보여준다.



루프는 HTTP와 HTTPS 설정뿐 아니라 로그인 상태를 판단하는 방식이나 캐시된 응답 때문에도 발생할 수 있다. 이번 글에서는 네 가지 사례를 통해 어떤 요청과 응답이 반복되는지 살펴본다. 등장하는 주소와 로그인 로직은 이해를 위한 예시이다.

리디렉션은 브라우저가 새 요청을 보내는 과정이다

예를 들어 서버가 다음과 같이 응답했다고 해보자.

```
HTTP/1.1 302 Found
Location: https://example.com/new
```

브라우저는 `Location`에 적힌 주소로 새 요청을 보낸다. 그곳에서 페이지를 받으면 이동이 끝나지만, 다시 이전 주소로 가라는 응답을 받으면 같은 과정을 반복하게 된다.

서로 다른 두 URL을 오가는 경우뿐 아니라, **이미 요청한 것과 똑같은 URL로 계속 이동하라는 응답을 받는 경우**도 루프가 될 수 있다. HTTPS 설정 사례가 여기에 해당한다.

301과 302 중 어느 코드를 썼느냐보다, 그 응답을 따라간 다음에도 다시 같은 이동이 발생하는지를 봐야 한다.

1. HTTPS로 접속했는데 서버는 HTTP 요청으로 판단하는 경우

웹서버 앞에 로드밸런서가 있고, 로드밸런서에서 HTTPS의 TLS 처리를 끝내는 구성을 가정해보자. 이를 **TLS 종료(TLS termination)**라고 한다. 여기서는 뒤에서 실제 페이지를 제공하는 웹서버를 **오리진(origin), 즉 원본 서버**라고 부르겠다.

통신 구간	사용하는 프로토콜
브라우저와 로드밸런서 사이	HTTPS
로드밸런서와 오리진 사이	HTTP

이 구성 자체가 리다이렉트 루프를 만드는 것은 아니다. 문제는 오리진이 자신에게 도착한 연결만 보고 다음 규칙을 적용할 때다.

> HTTP 요청이면 무조건 같은 주소의 HTTPS URL로 리디렉션한다.

브라우저가 `https://example.com`에 접속하면 다음 순서로 동작한다.

순서	동작
1	브라우저가 로드밸런서에 HTTPS로 요청한다.
2	로드밸런서가 오리진에는 HTTP로 요청을 전달한다.
3	오리진은 HTTP 연결을 보고 `301, Location: https://example.com`을 응답한다.
4	로드밸런서가 이 응답을 브라우저에 전달한다.
5	브라우저가 같은 HTTPS 주소로 다시 요청하면서 1번부터 반복된다.

브라우저는 이미 HTTPS로 접속했는데, 서버는 계속 HTTPS로 다시 접속하라고 하는 셈이다.

여기서 **브라우저가 HTTP와 HTTPS를 번갈아 사용하는 것은 아니다.** 브라우저는 계속 HTTPS를 사용하고 있고, HTTP인 구간은 로드밸런서와 오리진 사이다. 로드밸런서가 오리진에 HTTP로 요청을 전달하는 동작과, 브라우저에게 다른 URL로 이동하라고 응답하는 리디렉션을 구분해야 한다.

이 구성을 유지하려면 오리진이 사용자의 최초 접속 프로토콜을 알 수 있어야 한다. AWS ALB는 다음과 같은 헤더로 그 정보를 전달한다.

```
X-Forwarded-Proto: https
```

오리진의 웹서버나 애플리케이션이 이 헤더를 기준으로 판단하도록 설정하면, 내부 연결이 HTTP여도 사용자는 이미 HTTPS로 접속했다는 사실을 알 수 있다. 단, 이 값은 **신뢰할 수 있는 프록시가 설정한 값만 신뢰하도록** 구성해야 한다.

또는 오리진에도 인증서와 HTTPS 리스너를 준비하고, 로드밸런서가 오리진에 HTTPS로 연결하도록 구성할 수 있다. 어느 방식을 쓰든 오리진이 어떤 정보를 기준으로 HTTPS 여부를 판단하는지 확인해야 한다.

2. Cloudflare Flexible SSL과 오리진의 HTTPS 강제가 충돌하는 경우

Cloudflare는 웹사이트 앞에서 요청을 중계하며 CDN 등의 기능을 제공하는 서비스다. **Flexible SSL/TLS 모드**는 앞에서 살펴본 구성이 실제 서비스에서 나타나는 사례이다.

일반적인 443 포트의 HTTPS 접속을 기준으로 통신 구간은 다음과 같다.

통신 구간	Flexible 모드의 동작
브라우저와 Cloudflare 사이	HTTPS
Cloudflare와 오리진 사이	HTTP

예를 들어 브라우저가 `https://shop.example.com`에 접속하고, 오리진에는 HTTP 요청을 HTTPS로 돌려보내는 규칙이 있다고 해보자.

Cloudflare는 Flexible 설정에 따라 오리진에 HTTP로 요청한다. 오리진은 다음과 같이 응답한다.

```
HTTP/1.1 301 Moved Permanently
Location: https://shop.example.com
```

이 응답을 받은 브라우저는 같은 HTTPS 주소로 다시 접속한다. 하지만 Cloudflare는 오리진에 다시 HTTP로 요청하므로 같은 응답이 반복된다.

따라서 “Cloudflare와 오리진 사이에서 루프가 돈다”는 설명에는 **응답을 받은 브라우저가 다시 요청하는 과정**도 포함되어 있다. Cloudflare가 브라우저의 주소를 HTTP로 바꾸는 것은 아니다.

해결하려면 Cloudflare의 연결 방식과 오리진의 리디렉션 정책을 맞춰야 한다.

오리진에서 HTTPS를 제공할 수 있다면 인증서와 HTTPS 연결을 준비한 뒤, 오리진 인증서까지 검증하는 **Full (strict)** 모드를 사용할 수 있다. 이 모드는 인증서의 유효기간, 호스트 이름, 신뢰 가능한 발급자 등의 조건을 확인한다. 콘솔에서 모드만 바꾸기 전에 오리진이 해당 조건을 충족하는지 확인해야 한다.

Flexible 구성을 유지해야 한다면 오리진의 무조건적인 HTTP → HTTPS 리디렉션을 제거하거나 조건을 조정하고, 외부 사용자의 HTTPS 전환은 Cloudflare 앞단에서 처리하도록 역할을 맞출 수 있다. 다만 이 경우에도 Cloudflare와 오리진 사이가 HTTP라는 점은 그대로다.

3. 오래된 쿠키 때문에 로그인 상태를 서로 다르게 판단하는 경우

쿠키는 웹사이트가 브라우저에 저장하게 하는 작은 데이터이다. 브라우저는 이후 해당 사이트에 요청할 때 조건에 맞는 쿠키를 함께 보낸다.

로그인 상태를 관리하는 사이트에서는 쿠키에 **세션 ID**를 저장하기도 한다. 세션 ID는 서버가 보관한 로그인 정보를 찾는 식별자이다. 예를 들어 브라우저가 다음 값을 보내면 서버는 `abc123`에 대응하는 로그인 상태를 조회할 수 있다.

```
Cookie: session_id=abc123
```

브라우저에 쿠키가 남아 있다는 사실과 서버의 로그인 세션이 아직 유효하다는 사실은 서로 다르다. 쿠키 자체의 만료 시각이 지나지 않았더라도 서버에서는 해당 세션이 만료되거나 삭제됐을 수 있다.

다음은 **오래된 쿠키와 잘못된 로그인 처리 로직이 함께 존재하는 가상 예시**이다.

페이지	로그인 여부를 판단하는 방식	결과
`/mypage`	세션 ID로 서버의 로그인 정보를 조회	만료된 세션이므로 `/login`으로 302 리디렉션
`/login`	쿠키가 있는지만 확인하고 로그인했다고 잘못 판단	`/mypage`로 302 리디렉션

마이페이지는 "로그인이 만료됐으니 로그인 페이지로 가라"고 하고, 로그인 페이지는 "쿠키가 있으니 이미 로그인한 사용자"라며 마이페이지로 보낸다. 브라우저는 두 페이지를 계속 오가게 된다.

이 경우 사이트의 쿠키를 삭제하면 로그인 페이지가 더 이상 로그인했다고 오판하지 않아 루프가 멈출 수 있다. 하지만 근본적인 해결은 **로그인 페이지와 보호된 페이지가 동일한 기준으로 세션을 검증하게 만드는 것**이다. 정상적인 애플리케이션이라면 만료된 세션을 확인한 뒤 로그인 화면을 보여주면 된다.

따라서 오래된 쿠키가 있다는 이유만으로 반드시 루프가 발생하는 것은 아니다. 그 데이터를 처리하는 로직의 불일치가 함께 있어야 한다.

또한 쿠키는 브라우저가 직접 실행하는 리디렉션 명령이 아니다. 쿠키에 복귀할 URL을 저장하는 앱도 있지만, 그 값을 읽어 이동을 결정하는 것은 애플리케이션의 코드이다. 앞서 살펴본 **301 응답 캐시와 로그인 쿠키는 서로 다른 것**이다.

4. CDN에 남은 예전 리디렉션이 현재 서버 설정과 충돌하는 경우

CDN이나 캐싱 프록시는 원본 서버의 응답을 저장했다가 이후 요청에 대신 응답할 수 있다. 저장 대상은 이미지나 HTML만이 아니다. 설정에 따라 301 같은 리디렉션 응답도 캐시할 수 있다.

예를 들어 기존 안내 페이지 `/old`를 새 페이지 `/new`로 옮겼다고 해보자. 원본 서버는 `/old` 요청에 다음과 같이 응답했고, CDN은 캐시 정책에 따라 이 응답을 저장했다.

```
HTTP/1.1 301 Moved Permanently
Location: /new
Cache-Control: max-age=3600
```

이후 문제가 생겨 페이지를 원복했다고 가정해보자. 현재 원본 서버의 설정은 다음과 같다.

요청 경로	현재 원본 서버의 응답
`/old`	200 OK로 정상 페이지 제공
`/new`	`/old`로 301 리디렉션

원본 서버에서는 예전 규칙을 없앴지만, CDN에는 아직 캐시 유효기간이 남은 `/old → /new` 응답이 존재할 수 있다.

이때 `/new` 요청은 캐시에 없어 원본 서버까지 전달된다고 가정하면 다음과 같은 루프가 만들어진다.

순서	브라우저의 요청	응답을 제공한 곳	리디렉션 목적지
1	`/old`	CDN의 기존 캐시	`/new`
2	`/new`	원본 서버의 현재 설정	`/old`
3	`/old`	CDN의 기존 캐시	다시 `/new`

원본 서버를 수정해도 CDN이 저장된 응답을 재사용하면 그 변경을 바로 확인하지 않는다. 운영자는 설정을 원복했는데, 사용자는 계속 예전 이동 안내를 받는 상황이다.

CloudFront도 배포 설정과 응답 헤더에 따라 3xx 응답을 캐시한다. 301이라고 모든 CDN에서 같은 기간 동안 무조건 캐시되는 것은 아니므로 적용된 캐시 정책을 함께 봐야 한다.

이 경우에는 해당 경로의 CDN 캐시를 무효화하거나 삭제해야 한다. 이를 서비스에 따라 **Invalidation** 또는 **Purge**라고 부른다. 무효화가 적용된 뒤 다음 요청이 들어오면 원본 서버에서 현재 응답을 다시 받아올 수 있다.

브라우저 캐시와 CDN 캐시는 별개이다. 브라우저 캐시만 지워서는 CDN에 저장된 리디렉션 응답이 없어지지 않는다. 반대로 CDN 캐시를 무효화해도 브라우저에 남아 있는 별도의 리디렉션 캐시는 따로 확인해야 한다.

리다이렉트 루프를 확인할 때는 반복되는 URL과 `Location`을 먼저 살펴보고, **그 이동을 지시한 응답이 어디에서 만들어졌는지**를 추적해야 한다. 오리진이 방금 생성한 응답인지, CDN에 저장된 이전 응답인지, 로그인 상태에 따라 애플리케이션이 만든 응답인지에 따라 확인할 곳이 달라진다.